BRAC UNIVERSITY
Department of Computer Science and Engineering
CSE220: Data Structures

A

Examination: Final
Duration: 1 Hour 40 Minutes

Semester: Fall 2025
Full Marks: 35
No. of Pages: 3

Name: **(Please write in CAPITAL LETTERS)**	**ID:**	**Section:**

✔ **Answer all questions. No washroom breaks. Understanding questions is part of the Exam.**
✔ At the end of the exam, put the question **paper** inside the answer script and **return both**.

Question 1: CO3 [10 Points]

i) A perfect binary tree of height 5, how many nodes are there in the left subtree of the root? (the height of a root node without childs is 0)
[1 Mark]

a) 23 b) 7 c)25 d)31

ii) In the sink/heapify operation of the remove function in a Max-Heap, with which child is the value of a node compared? [1 Mark]
a) Left child b) Right child
c) The greater among its two children
d) The minimum among its two children

iii) What is the maximum number of edges in a directed graph with 6 vertices(no self-loops)?
[1 Mark]
a) 15 b) 36 c) 30 d) 12

iv) A directed graph is represented using an adjacency list. If the total number of nodes in full adjacency lists is 15, how many edges are present? [1 Mark]
a) 5 b) 10 c) 15 d) Cannot be determined

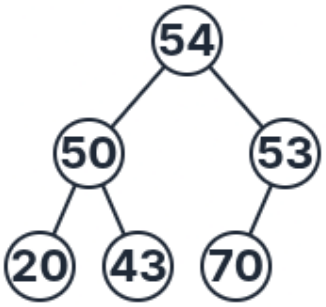
v) A binary tree represented using an array of size 72. Assume, all indices of the array have values except for 0 index. Is the node at index 37 a leaf node?
a) Yes
b) No

vi) Adjacency matrix representation [2 Marks]

	A	B	C	D
A	0	1	0	0
B	0	0	1	0
C	0	0	0	0
D	1	0	1	0

Draw the graph and identify a vertex from which all other vertices can be visited.

vii) Which of the following tasks cannot be performed on a singly linked list in O(1) space and O(n) time complexity? [1 Mark]
a) Finding the k-th node from the end of the list
b) Finding the middle node of the list
c) Detecting whether the list contains a cycle
d) Checking whether the list is a palindrome



viii) You are given a Max-heap tree, but due to memory corruption, a node is violating the max-heap property.

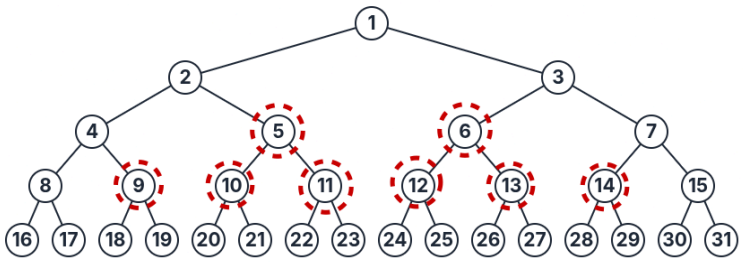
Using the concepts of Swim() or Sink() (to move a value up or down the heap), update the tree so that it becomes a valid max-heap again .
Note: You are not allowed to remove any elements from the tree. No need to write any codes. [2 Marks]

Question 2: CO4 [8 Points]

You are given the root of a perfect binary tree as a parameter in a function/method called **outerInnerSum()**. Your task is to **print** out two different summations. First summation would be of all the nodes which are the exterior of this BinaryTree (nodes which are NOT marked using dashed circle) then the second summation would be of all the nodes which are interior of this Binary (nodes which are marked in dashed circle).

NOTE: You may use multiple helper methods/functions to solve this problem. Consider that BTNode class is already implemented with the instance variables elem (integer), left (BTNode), right (BTNode).

You cannot use any extra data structures to solve this problem.

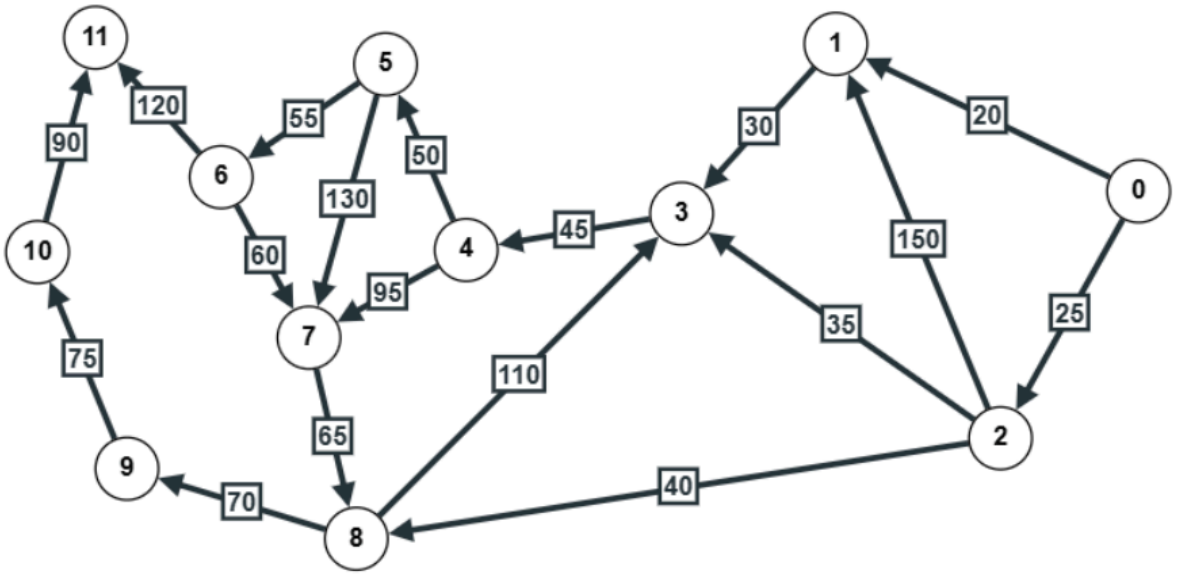
<table>
<tr><th>Sample Given Tree</th><th>Sample Method Call</th><th>Sample Output</th></tr>
<tr><td rowspan="2"> 
</td><td><code>outerInnerSum(root)</code></td><td><code>Exterior: 416
Interior: 80</code></td></tr>
<tr><td colspan="2">Explanation:
Exterior sum= (1+2+3+4+....+31) = 416.
Interior sum= (5+6+9+10+11+12+13+14) = 80</td></tr>
<tr><th>Java function Signature</th><th colspan="2">Python function Signature</th></tr>
<tr><td><pre>static void outerInnerSum(BTNode root) {
  // Write your code here
}</pre></td><td colspan="2"><pre>def outerInnerSum(root):
  # Write your code here</pre></td></tr>
</table>

Question 3: CO2 [4 Points]

Consider the given **directed weighted graph**, where each vertex is labeled with a numerical value, and each directed edge has an associated weight. You are required to perform a **special Breadth-First Search (BFS)** traversal starting from **vertex 5**. In this special BFS, when a vertex has more than one neighbor, the next vertex is chosen based on the smallest sum of the vertex value and the edge weight. In other words, the neighbor with the minimum (vertex value + edge weight) is visited first.

You must (a) **show the step-by-step simulation** of the traversal. After completing the traversal, write the (b) **final BFS traversing order** and (c) **list down any vertices that remain unvisited.**

You are **not required to write any code** for this problem. All the necessary steps of the simulation must be shown.



Question 4: CO1 [5 Points]

Your task is to implement a method/function **scanMaxInGroup(numbers, k)** that:

- takes an **array of integers** named **numbers**, where elements are stored in random order.
- takes an integer **k**, representing how many integers you need to group.
- returns **an array** containing the maximum numbers for each group in sequential order.

A **Max Heap** class is already provided with all required functionality. You can use its public methods insert() and extract() after creating its object.

Note: You must use **MaxHeap** to solve the problem. You're allowed to create only 1 integer array.

Given parameters & Method/Function Call	Sample Returned Array	Explanation
numbers = [1, 3, -1, -3, 5, 3, 6] k = 3 scanMaxInGroup(numbers, k)	[3, 3, 5, 5, 6]	The radar observes 3 numbers at a time and records the highest number for each observation: [1, 3, -1] → 3, [3, -1, -3] → 3, [-1, -3, 5] → 5, [-3, 5, 3] → 5, [5, 3, 6] → 6
numbers = [2, 1, 5, 3, 4, 6] k = 4 scanMaxInGroup(numbers, k)	[5, 5, 6]	The radar observes 4 numbers at a time and records the highest number for each observation: [2, 1, 5, 3] → 5, [1, 5, 3, 4] → 5, [5, 3, 4, 6] → 6

Java Method Signature	Python Function Signature

`static int[] scanMaxInGroup(int[] numbers, int k){`
`  //TODO`
`}`
	`def scanMaxInGroup(numbers, k):`
	`  #TODO`

Question – 5: CO2 [8 Points]

An airline plans its flight transfer network using a directed graph, where each vertex represents an airport and a directed edge A → B indicates a direct flight from airport A to airport B. The network is implemented using an adjacency list representation, where each vertex stores a linked list of Edge objects.

Write a method named **networkCheck()** that computes the in-degree and out-degree of each airport and stores the results in a 2D integer array of n (number of vertices) rows & 2 columns, where [i][0] represents the in-degree and [i][1] represents the out-degree of airport *i*. The method should then determine whether the network is a regular directed graph (i.e. all airports have the same in-degree and the same out-degree) and **print** accordingly and **return** the 2D integer array containing the in-degree & out-degree of all vertices.

Assume, the Edge class is given which contains the variables source (int), destination (int), and next (Edge).

Sample Input1 (network)	Output	Explanation
0: (0,1) → (0,2) → (0,3), 1: (1,0) → (1,2) → (1,3), 2: (2,0) → (2,1) → (2,3), 3: (3,0) → (3,1) → (3,2)	Regular graph **Returned 2DArray:** {{3, 3}, {3, 3}, {3, 3}, {3, 3} }	Every airport has exactly three incoming flights and three outgoing flights. Since the in-degree and out-degree are identical for all airports, the directed flight network is regular.
0: (0,1) → (0,2), 1: (1,2) → (1,3), 2: (2,3), 3: (3,4), 4: (4,1)	Not regular graph **Returned 2DArray:** {{0, 2},{2, 2},{2, 1},{2, 1}, {1, 1}}	The number of incoming and outgoing flights varies across airports. For example, airport 0 has an out-degree of 2 but an in-degree of 0, while airport 4 has an in-degree of 1 and an out-degree of 1. Therefore, the network is not regular.

Python Function Signature	Java Method Signature
`def networkCheck(network):` `  #To do`	`static int[][] networkCheck(Edge[] network){` `  #To do}`